

Taller 3  
Programación Discreta

Julián Camilo Hortúa Franco

Matemáticas Discretas I  
Profesor: Jhoan Sebastian Tenjo Garcia

29 de Julio de 2026

Universidad Nacional de Colombia  
Sede Bogotá

## Introducción

Este documento presenta las soluciones de los problemas del Taller 3 de Matemáticas Discretas I, el cual corresponde a varios temas del curso como criptografía, grafos, álgebra de Boole, entropía de Shannon y una simulación básica de un qubit, la idea es mostrar que cada programa se resuelve usando un concepto de matemáticas discretas concreto que lo hace funcionar: una operación modular, un algoritmo sobre un grafo, una tabla de verdad, una fórmula de entropía. Por eso, en cada uno de los 10 ejercicios se explica qué problema resuelve, qué idea matemática usa, cómo se implementó, qué pruebas se le hicieron y qué limitaciones tiene.

### Problema 1. Cifrado César: romper un mensaje antiguo

**Problema:** Cifrar y descifrar un texto usando el cifrado César con un desplazamiento  $k$ , y además poder recuperar el mensaje original cuando no se conoce  $k$ , probando todos los desplazamientos posibles.

**Idea Matemática:** Cada letra se representa por su posición en el alfabeto (0 a 25) y el cifrado consiste en desplazar esa posición sumando  $k$  módulo 26:  $(\text{posición} + k) \bmod 26$ . El módulo es lo que hace que el desplazamiento dé la vuelta al llegar a la Z sin salirse del alfabeto. Descifrar es la operación inversa, restando  $k$  en vez de sumarlo.

El descifrado usa el desplazamiento contrario porque para descifrar debe deshacer lo que hizo el cifrado: si cifrar suma  $k$  posiciones, descifrar debe restar esas mismas  $k$  posiciones para volver al punto de partida. El ataque de fuerza bruta es posible porque el espacio de claves es extremadamente pequeño: solo existen 26 desplazamientos posibles, así que probarlos todos y leer cuál produce texto con sentido toma poco tiempo y gasta pocos recursos, sin necesidad de conocer  $k$  de antemano.

**Solución:** Se representan las letras en dos listas (mayúsculas y minúsculas) y se usa el índice de cada letra en su lista para calcular el corrimiento. La función `cifrar()` recorre el mensaje carácter por carácter: si es letra, aplica  $(\text{posición} + k) \% 26$ ; si no (espacios, números, signos), lo deja igual. `descifrar()` hace lo mismo, pero restando  $k$ . La función `no_k()` prueba los 26 desplazamientos posibles llamando a `descifrar` con cada valor de  $k$ , para poder identificar el mensaje original a simple vista cuando no se conoce la clave.

**Pruebas Realizadas:** Se probaron cuatro casos: el caso obligatorio del taller (HOLA UNAL con  $k=3$ , verificando que el resultado es KROD XQDO), un caso con minúsculas, un caso con caracteres especiales y números mezclados con letras para confirmar que se preservan sin alterar, y un caso de fuerza bruta probando los 26 valores de  $k$  sobre un mensaje cifrado para verificar que entre los resultados aparece el mensaje original.

**Limitaciones:** El programa solo maneja el alfabeto latino sin la letra Ñ; cualquier otro carácter se deja igual en vez de cifrarse. Además, la fuerza bruta funciona porque solo hay 26 claves posibles: con un espacio de claves más grande este método dejaría de ser práctico.

### Problema 2. RSA de juguete: llaves, cifrado y descifrado

**Problema:** Implementar una versión pequeña de RSA que, a partir de dos primos  $p$ ,  $q$  y un exponente público  $e$ , calcule la llave privada y permita cifrar y descifrar un mensaje  $M$

**Idea Matemática:** RSA se basa en aritmética modular. Se calcula  $n = p \cdot q$  y  $\phi(n) = (p-1)(q-1)$ . El exponente privado  $d$  es el inverso multiplicativo de  $e$  módulo  $\phi(n)$ , es decir, el número que cumple  $e \cdot d \equiv 1 \pmod{\phi(n)}$ . Ese inverso solo existe si  $e$  y  $\phi(n)$  son primos relativos ( $\gcd(e, \phi(n)) = 1$ ); si no lo son, no hay llave privada válida. El cifrado y el descifrado son potenciación modular:  $C \equiv M^e \pmod{n}$  y  $M \equiv C^d \pmod{n}$ .

Los primos  $p$  y  $q$  son la base de la seguridad de RSA:  $n = p \cdot q$  es fácil de calcular, pero factorizarlo de vuelta en  $p$  y  $q$  es difícil si los primos son grandes, y de esos primos depende  $\phi(n)$ , que a su vez determina la llave privada  $d$ . El

inverso modular es lo que conecta la llave pública con la privada: cifrar eleva a la  $e$  y descifrar eleva a la  $d$ , y como  $d$  fue construido para que  $e \cdot d \equiv 1 \pmod{\phi(n)}$ , al combinar ambas operaciones se regresa exactamente al mensaje original. Todo esto funciona gracias al teorema de Euler sobre congruencias, que garantiza que elevar un número a un exponente congruente con 1 módulo  $\phi(n)$  devuelve el número original módulo  $n$ .

**Solución:** El inverso modular  $d$  se calcula con el algoritmo de Euclides extendido (`euclides_ext`), que además de dar el máximo común divisor entre dos números, devuelve los coeficientes de Bézout necesarios para obtener el inverso. La función `calcular_d()` usa ese resultado y valida que  $\gcd(e, \phi(n)) = 1$ ; si no se cumple, lanza un error indicando que  $e$  no es válido. cifrado y descifrado aplican directamente las fórmulas de potenciación modular usando el operador `%` de Python.

**Pruebas Realizadas:** Se probó el caso obligatorio del taller ( $p=61$ ,  $q=53$ ,  $e=17$ ,  $M=65$ ), confirmando que se obtienen exactamente los valores esperados:  $n=3233$ ,  $\phi(n)=3120$ ,  $d=2753$ ,  $C=2790$  y que al descifrar se recupera  $M=65$ . También se probó un caso con  $e$  inválido ( $e=15$ , que no es primo relativo con  $\phi(n)=3120$ ), verificando que el programa lanza el error correspondiente en vez de calcular un  $d$  incorrecto. Por último, se probó un caso límite con  $M=1$ , para confirmar que el cifrado y descifrado funcionan también en los extremos del rango de mensajes.

**Limitaciones:** El programa funciona bien para primos pequeños como los del taller, pero no está optimizado para primos grandes así que con números del tamaño usado en RSA real sería demasiado lento.

### Problema 3. MPC básico: calcular un promedio sin mostrar los datos

**Problema:** Simular un protocolo de cómputo multipartito seguro que permita calcular la suma y el promedio de un conjunto de notas, repartiendo la información entre tres servidores sin que ninguno tenga acceso a las notas originales.

**Idea Matemática:** El protocolo se basa en aritmética modular y en la idea de repartir un dato en partes que solo tienen sentido si se combinan. Cada nota  $x$  se descompone en tres partes  $s_1$ ,  $s_2$ ,  $s_3$  tales que  $x \equiv s_1 + s_2 + s_3 \pmod{M}$ , generando  $s_1$  y  $s_2$  de forma aleatoria y despejando  $s_3$  para que la congruencia se cumpla. Individualmente, cada parte parece un número sin relación con  $x$  y solo al sumar las tres partes módulo  $M$  se recupera la información original, que para este caso es la suma de las notas.

Por ejemplo, si  $x = 40$  y  $M = 1000003$ , se pueden generar  $s_1 = 700000$  y  $s_2 = 250000$  al azar, y calcular  $s_3 = (40 - 700000 - 250000) \% 1000003 = 50003$ . Ninguno de esos tres números por separado da ninguna pista sobre que la nota original era 40, podrían corresponder a cualquier nota, dependiendo de qué valores aleatorios se hayan usado. Solo al sumar los tres y reducir módulo  $M$  se recupera 40. Por eso cada servidor puede tener su parte sin conocer la nota real, y solo al combinar las tres partes se obtiene información útil.

**Solución:** `dividir_nota()` genera  $s_1$  y  $s_2$  aleatorios en el rango  $[0, M-1]$  y calcula  $s_3 = (x - s_1 - s_2) \% M$ . `repartir_notas()` aplica esto a cada nota de la lista y distribuye cada parte en su servidor correspondiente. `calcular_suma()` suma todos los valores de cada servidor por separado y luego combina esas tres sumas parciales módulo  $M$ , obteniendo la suma total original sin que ningún servidor haya conocido las notas individuales. `calcular_promedio()` simplemente divide esa suma entre el número de notas.

**Pruebas Realizadas:** Se probó el caso obligatorio del taller ( $[40, 35, 50, 25]$ ) verificando que la suma reconstruida es 150 y el promedio 37.5. Se probó un caso con notas en los valores extremos del rango permitido entre 0 y 50 para verificar que el protocolo funciona igual de bien en los límites. Y se probó un caso con una lista más grande de notas de 10 estudiantes verificando que el protocolo escala bien a una mayor cantidad de datos

**Limitaciones:** El programa asume que los tres servidores no se comunican entre sí ni comparten sus partes, si dos servidores se pusieran de acuerdo, podrían reconstruir la nota original, esto es algo que evita en los MPC reales.

#### **Problema 4. Ruta más corta: una ciudad como grafo**

**Problema:** Implementar el algoritmo de Dijkstra para encontrar la ruta más corta entre dos vértices de un grafo ponderado, devolviendo tanto la distancia total como la secuencia de vértices que forman esa ruta.

**Idea Matemática:** El grafo se representa como un diccionario de diccionarios, esto es una lista de adyacencia con pesos. Dijkstra construye las distancias mínimas desde el origen de forma incremental: en cada paso elige el vértice no visitado con menor distancia acumulada, y desde ahí actualiza las distancias de sus vecinos: si pasar por el vértice actual da una ruta más corta que la que tenían antes, se actualiza. Esto funciona porque, al elegir siempre el vértice no visitado con menor distancia, se garantiza que esa distancia ya es la mínima posible y no cambiará después.

El algoritmo asume que una vez que un vértice es marcado como visitado, su distancia ya es la mínima posible y no se volverá a mejorar. Eso solo es cierto si todos los pesos son no negativos, porque un peso negativo podría, en teoría, encontrar después una ruta más corta pasando por un vértice ya cerrado, rompiendo la lógica del algoritmo. Que un camino sea óptimo significa que es la de menor suma total de pesos entre todas las secuencias posibles de vértices que conectan el origen con el destino, ósea, es el camino que requiere un menor número de pasos, puede haber varios caminos óptimos y el programa elegirá el primero que encuentre.

**Solución:** Se inicializan todas las distancias en infinito excepto el origen, que empieza en 0. En cada iteración se busca el vértice no visitado con menor distancia (vertice\_actual), se marca como visitado y se revisan sus vecinos: si la distancia acumulada hasta el vecino a través de vertice\_actual es menor que la que tenía registrada, se actualiza y se guarda en el diccionario previo desde dónde se llegó. Al terminar, la ruta se reconstruye recorriendo (previo) hacia atrás desde el destino hasta el origen.

**Pruebas Realizadas:** Se probó un grafo pequeño de 4 vértices para verificar el comportamiento básico del algoritmo. Se probó el caso obligatorio con un grafo de 8 estaciones y 12 conexiones simulando una red de TransMilenio, confirmando distancia y ruta entre los extremos de la red y se probó un caso límite con estaciones desconectadas, verificando que cuando no existe camino posible, la función devuelve distancia infinita y una ruta que contiene únicamente el destino.

**Limitaciones:** La búsqueda del vértice no visitado con menor distancia se hace recorriendo todos los vértices en cada iteración, en vez de usar una cola de prioridad, esto sirve para grafos pequeños como los del taller, pero sería costoso en grafos grandes. Además, el algoritmo asume pesos no negativos, entonces si el grafo tuviera pesos negativos se rompería la lógica del program.

#### **Problema 5. Cierre de una estación: medir el impacto en la red**

**Problema:** Medir el impacto de cerrar un vértice de la red para varios pares origen-destino calculando cómo cambia la distancia más corta antes y después del cierre y detectar cuándo un par queda desconectado.

**Idea Matemática:** Es una aplicación directa de Dijkstra, eliminar un vpetrice con todas sus conexiones provoca que los caminos más cortos entre dospuntos cambien, la idea matemática consiste en calcular Dijkstra al grafo completo para luego eliminar un vértice jun a sus conexiones para usar nuevamente Dijkstra sobre este grafo incompleto y comparar los resultados para el camino más corto en ambos casos.

Para este problema se cerraron dos vértices distintos de la red de TransMilenio para comparar: uno con varias conexiones que actúa como nodo de paso entre varias estaciones y otro nodo intermedio. Un cierre produce un

impacto importante cuando el vértice cerrado era el único puente entre dos zonas de la red, en ese caso algunos pares quedan desconectados o su distancia aumenta mucho porque deben rodear por un camino más largo. En cambio, si existen rutas alternas con distancias similares, el cierre apenas cambia la distancia total, como se observó en el caso de la red con camino alterno.

**Solución:** `cerrar_vertice()` genera una copia del grafo con `copy.deepcopy` para no alterar el original y elimina el vértice indicado incluyendo las referencias a él en las listas de adyacencia de sus vecinos. `comparar_cierre()` recorre una lista de pares origen-destino: para cada uno calcula la distancia con Dijkstra en el grafo original y en el grafo con el vértice cerrado y según el resultado clasifica el estado como "Conectado" mostrando la diferencia de distancia, "Desconectado" si la distancia después es infinita o "Vértice cerrado" si el propio origen o destino del par es el vértice que se cerró.

**Pruebas Realizadas:** Se probó el cierre de un nodo con varias conexiones dentro de la red de TransMilenio, sobre tres pares origen-destino. Se probó también el cierre de otro nodo intermedio distinto, sobre los mismos tres pares, para comparar el impacto relativo de cerrar un nodo u otro. Y se probó un tercer caso con una red más pequeña que tiene un camino alterno explícito, verificando que al cerrar el nodo del camino más corto, el algoritmo encuentra correctamente la ruta alterna en vez de desconectar el par.

**Limitaciones:** El programa evalúa el cierre de un solo vértice a la vez y no tiene ninguna forma de implementar cierres simultáneos de varios nodos o aristas

## Problema 6. Coloreo de grafos: organizar exámenes sin choques

**Problema:** Asignar colores a los vértices de un grafo de conflictos de forma que ningún par de vértices adyacentes comparta el mismo color.

**Idea Matemática:** El coloreo de grafos modela situaciones donde elementos en conflicto conectados por una arista no pueden compartir un mismo recurso, en este caso se tiene el caso de una franja horaria representada por un color. Se usa un algoritmo voraz que se recorre los vértices y a cada uno se le asigna el primer color disponible que ninguno de sus vecinos ya tenga asignado.

El algoritmo solo mira los colores que ya usaron los vecinos hasta ese momento y elige el primero disponible, no analiza el grafo completo ni prueba distintas combinaciones para encontrar el mínimo número posible de colores. Esto significa que dependiendo del orden de los vértices puede terminar usando más colores de los necesarios. Sin embargo, sí garantiza que la asignación es válida, porque en cada paso revisa explícitamente los colores de los vecinos ya coloreados y nunca elige uno que esté en conflicto con ellos.

**Solución:** `colorear_grafo()` recorre los vértices del grafo, para cada uno arma el conjunto de colores ya usados por sus vecinos y le asigna el primer entero que no esté en ese conjunto. `verificar_coloreo()` recorre todas las aristas y confirma que ningún par de vértices adyacentes tenga el mismo color devolviendo False si encuentra un conflicto. `resumen_coloreo()` agrupa los vértices por color asignado y muestra cuántos colores se usaron en total y qué vértices quedaron en cada uno.

**Pruebas Realizadas:** Se probó un grafo de conflicto con 4 vértices. Se probó un triángulo  $K_3$  que matemáticamente requiere exactamente 3 colores para verificar que el algoritmo devuelve ese número. Y se probó un caso límite con vértices totalmente aislados, sin ninguna arista, donde se espera que todos puedan compartir el mismo color al no tener conflictos entre sí. En los tres casos se usó `verificar_coloreo` para confirmar que la asignación resultante es válida.

**Limitaciones:** Como ya se mencionó, al ser un algoritmo voraz, el número de colores usado depende del orden en que se recorren los vértices, un orden distinto podría dar un resultado con más o menos colores para el mismo grafo.

## Problema 7. Tablas de verdad y circuitos lógicos

**Problema:** Generar la tabla de verdad de expresiones booleanas con varias variables y conectivos lógicos (AND, OR, NOT, XOR), y además permitir evaluar cada expresión en una entrada concreta.

**Idea Matemática:** Una expresión booleana con  $n$  variables tiene exactamente  $2^n$  combinaciones posibles de entrada donde cada variable puede ser 0 o 1. La tabla de verdad enumera todas esas combinaciones y muestra el resultado de la expresión en cada una describiendo por completo el comportamiento lógico de la función.

Cada fila de la tabla de verdad corresponde a una posible combinación de señales de entrada a un circuito digital, y el resultado de esa fila es la señal de salida que produciría el circuito para esa combinación. Los operadores lógicos (AND, OR, NOT, XOR) usados en la expresión corresponden directamente a compuertas lógicas físicas; esto significa entonces que una expresión booleana muestra cómo se deben conectar las compuertas para contruir un circuito que devuelva la salida que muestra la tabla.

**Solución:** Cada expresión se define como una función (expr1, expr2, expr3) que traduce directamente la fórmula booleana usando los operadores lógicos de Python (and, or, not, y != para representar XOR entre valores booleanos). tabla\_verdad() usa itertools.product([0,1], repeat=n) para generar automáticamente las  $2^n$  combinaciones de entrada, evalúa la función en cada una y va imprimiendo cada fila junto con el resultado. evaluar() simplemente llama a la función con valores concretos, para probar un caso puntual sin generar toda la tabla.

**Pruebas Realizadas:** Se generaron las tablas de verdad completas de las tres expresiones obligatorias del taller:  $(A \wedge B) \vee \neg C$ ,  $(A \oplus B) \wedge C$  y  $(A \vee B) \wedge (\neg A \vee C)$ . Además se probó cada expresión de forma puntual en los dos casos extremos: todas las variables en 0 y todas las variables en 1 para verificar el comportamiento en los bordes del dominio.

**Limitaciones:** No se puede ingresar una fórmula arbitraria en tiempo de ejecución. Además, el programa solo soporta hasta 3 variables en las expresiones probadas.

## Problema 8. Simplificación booleana: hacer un circuito más barato

**Problema:** obtener una expresión simplificada en forma suma de productos que use el menor número de literales posible a partir de los minitérminos de una función booleana de 3 o 4 variables,, y verificar que sea equivalente a la función original.

**Idea Matemática:** Idea matemática. Un minitérmino representa una combinación de entrada donde la función vale 1, codificada como un número entero cuya representación binaria indica el valor de cada variable. La simplificación se basa en el algoritmo de Quine-McCluskey: dos términos que difieren en exactamente un bit se pueden combinar en uno solo, reemplazando ese bit por un guion (-), que indica que esa variable no afecta el resultado (es la aplicación directa de la ley  $X \cdot A + X \cdot \neg A = X$  del álgebra de Boole). Repitiendo este proceso de combinación hasta que ya no se puedan agrupar más términos, se obtienen los implicantes primos, que forman la expresión simplificada.

Un mintermino es una combinación específica de valores de entrada (0s y 1s) para la cual la función booleana da como resultado 1; representa un caso exacto en el que la función se activa. Dos expresiones son equivalentes si producen el mismo resultado para absolutamente todas las combinaciones posibles de entrada, es decir, si sus tablas de verdad son idénticas fila por fila sin importar que la forma algebraica de las expresiones sea distinta, porque lo que define a una función booleana es su comportamiento, su tabla, no la manera particular en que está escrita.

**Solución:** `a_binario()` convierte cada minitérmino a su representación binaria con ceros a la izquierda según el número de variables. `difieren_en_un_bit()` compara dos términos y determina si solo cambian en una posición. `combinar()` une dos términos que difieren en un bit, poniendo un guion en esa posición. `simplificar_minterminos()` aplica esto repetidamente: en cada ronda intenta combinar todos los pares de términos, guarda los que no se pudieron combinar (implicantes primos definitivos) y continúa con los nuevos términos combinados, hasta que no hay más combinaciones posibles. `termino_a_texto()` y `expresion_simplificada_texto()` traducen los términos binarios con guiones a una expresión legible en `and/or/not`. Para verificar la equivalencia, `evaluar_por_minterminos()` reconstruye la tabla de verdad original a partir de la lista de minitérminos, `evaluar_implicantes` evalúa la expresión simplificada, y `verificar_equivalencia()` compara ambas en las  $2^n$  combinaciones posibles de entrada.

**Pruebas Realizadas:** Se probó el caso sugerido del taller con minitérminos impares de 3 variables ( $\{1,3,5,7\}$ ), donde la simplificación esperada equivale a la variable C. Se probó un caso de 4 variables con minitérminos pares ( $\{0,2,4,6,8,10,12,14\}$ ), que corresponde a una función donde la última variable siempre vale 0. Y se probó un caso límite donde todos los minitérminos posibles de 3 variables están activos ( $\{0,\dots,7\}$ ), es decir, la función es siempre verdadera, para confirmar que el algoritmo la simplifica correctamente a una expresión constante. En los tres casos se usó `verificar_equivalencia` para confirmar que la expresión simplificada produce exactamente la misma tabla de verdad que los minitérminos originales.

**Limitaciones:** El algoritmo implementado agrupa por diferencia de un bit en todas las rondas, pero no aplica el paso final de selección de implicantes primos esenciales mediante una tabla de cobertura; en funciones más complejas esto podría dejar implicantes primos redundantes en la expresión final, aunque siga siendo equivalente a la función original.

## Problema 9. Shannon: medir información en un mensaje

**Problema:** Calcular cuánta información o incertidumbre tiene un texto, comparar la entropía entre dos mensajes distintos, y como parte opcional construir un código de Huffman para el texto y comparar su longitud promedio con la entropía teórica.

**Idea Matemática:** La entropía de Shannon mide la incertidumbre promedio de una fuente de símbolos:  $H = -\sum p_i \cdot \log_2(p_i)$ , donde  $p_i$  es la probabilidad de cada símbolo. Un texto muy repetitivo tiene entropía baja, porque es fácil predecir el siguiente símbolo; un texto con símbolos muy variados y parejos en frecuencia tiene entropía alta, porque hay más incertidumbre sobre cuál vendrá. Como extensión, el código de Huffman construye una codificación binaria óptima asignando códigos más cortos a los símbolos más frecuentes, y su longitud promedio siempre es mayor o igual a la entropía teórica (el límite de Shannon).

La fórmula depende de las probabilidades de los símbolos del texto y no de la longitud del mismo, un texto con 1000 A's tiene entropía 0 porque siempre se sabe cual es el próximo carácter y en cambio, un texto mucho más corto pero con caracteres más variados tiene una entropía alta porque es difícil predecir el carácter que sigue; por eso la entropía mide la incertidumbre y no la longitud del texto.

**Solución:** `frecuencias()` cuenta cuántas veces aparece cada símbolo con Counter. `Probabilidades()` divide cada frecuencia entre el total de símbolos. `Entropía()` aplica directamente la fórmula de Shannon sobre esas probabilidades. `analizar_texto()` junta estos tres pasos y muestra un resumen. `Comparar()` calcula la entropía de dos textos y determina cuál es más impredecible. Para la extensión de Huffman: `NodoHuffman` representa los nodos del árbol binario; `arbol_huffman()` construye el árbol fusionando iterativamente, mediante una cola de prioridad (heapq), los dos símbolos de menor frecuencia hasta quedar un solo árbol; `generar_codigos()` recorre el árbol asignando un código binario a cada símbolo según el camino (0 = izquierda, 1 = derecha);

longitud\_promedio\_huffman calcula la longitud esperada del código ponderando por la frecuencia de cada símbolo.

**Pruebas Realizadas:** Se comparó un texto totalmente repetitivo ("AAAAAAAAA") contra una frase variada con muchos símbolos distintos, confirmando que el segundo tiene mayor entropía. Se analizó la eficiencia de Huffman en ambos escenarios: el de entropía nula y el de alta entropía, verificando que la longitud promedio del código siempre es mayor o igual a la entropía teórica. Y se probó un caso límite con una cadena binaria perfectamente balanceada y periódica ("01010101010101"), que representa el caso de máxima incertidumbre entre solo dos símbolos.

**Limitaciones:** El cálculo de entropía trata el texto como una secuencia de símbolos independientes y no considera la entropía patrones entre símbolos consecutivos, por lo que un texto con estructura repetitiva pero símbolos individualmente variados (como "ABABAB...") podría dar una entropía más alta de lo que percibe a simple vista.

### Problema 10. Primer simulador cuántico: bits, qubits y mediciones

**Problema:** Simular un qubit representado como un vector de dos amplitudes, permitiendo aplicar las compuertas cuánticas X, Z y H, calcular las probabilidades de medir 0 o 1, y simular 1000 mediciones reportando las frecuencias observadas.

**Idea Matemática:** Un qubit se representa como una combinación  $\alpha|0\rangle + \beta|1\rangle$ , donde  $\alpha$  y  $\beta$  son amplitudes de probabilidad tales que  $|\alpha|^2 + |\beta|^2 = 1$ . Aplicar una compuerta cuántica equivale a multiplicar el vector de estado por la matriz correspondiente a esa compuerta. La probabilidad de medir 0 es  $|\alpha|^2$  y la de medir 1 es  $|\beta|^2$ ; al medir, el estado colapsa a uno de los dos valores según esas probabilidades, lo cual se puede simular con muestreo aleatorio.

En este simulador las probabilidades se calculan con álgebra lineal clásica multiplicando matrices y vectores con números normales y luego se simula el colapso del estado usando un generador de números pseudoaleatorios clásico para decidir si el resultado es 0 o 1. En un computador cuántico real el colapso es un fenómeno físico donde el estado del qubit, que de verdad existe en superposición, se resuelve de forma inherentemente probabilística al medirlo sin que exista un valor definido de antemano ni un algoritmo clásico detrás decidiéndolo. La no es simulada por un programa, es un fenómeno físico existeste.

**Solución:** El estado se representa como una lista [alpha, beta], y ESTADO\_0/ESTADO\_1 son los estados base  $|0\rangle$  y  $|1\rangle$ . Las compuertas X, Z y H están definidas como matrices 2x2. aplicar\_compuerta() hace la multiplicación matriz-vector manualmente para transformar el estado. probabilidades\_eleva al cuadrado cada amplitud para obtener P(0) y P(1). simular\_mediciones() genera 1000 números aleatorios y cuenta cuántos caen por debajo de p0, simulando así el colapso probabilístico del estado. mostrar\_estado() junta todo: muestra el estado, sus probabilidades analíticas, y el resultado de las 1000 mediciones simuladas.

**Pruebas Realizadas:** Se aplicaron las tres compuertas (X, Z, H) sobre el estado  $|0\rangle$ , y además se aplicó H dos veces seguidas para observar la involución. Se verificaron los tres casos obligatorios del taller con assert: que  $X|0\rangle$  sea exactamente  $|1\rangle$ ; que  $H|0\rangle$  produzca una superposición con probabilidades de 50%/50% (con una tolerancia de  $1e-9$  para errores de precisión numérica); y que  $HH|0\rangle$  regrese al estado original  $|0\rangle$ , confirmando que H es su propia inversa.

**Limitaciones:** El simulador solo maneja amplitudes reales, no complejas; esto es suficiente para las compuertas X, Z y H usadas en el taller, pero no representaría correctamente compuertas cuánticas que sí requieren números complejos (como la compuerta S o T). Además, al ser una simulación clásica con listas, no hay entrelazamiento ni múltiples qubits: solo se modela un qubit aislado.